

These two string objects have different identities (40718944 and 40719224) because they're different objects. But variables that refer to mutable objects can have their values modified *in-place*. For example, enter the following into the interactive shell:

---

```
>>> spam = ['cat', 'dog']
>>> id(spam)
33805576
❶ >>> spam.append('moose')
❷ >>> spam[0] = 'snake'
>>> spam
['snake', 'dog', 'moose']
>>> id(spam)
33805576
```

---

The `append()` method ❶ and item assignment by indexing ❷ both modify the value of the list in-place. Even though the list's *value* has changed, its *identity* remains the same (33805576). But when you concatenate a list using the `+` operator, you create a new object (with a new identity) that overwrites the old list:

---

```
>>> spam = spam + ['rat']
>>> spam
['snake', 'dog', 'moose', 'rat']
>>> id(spam)
33840064
```

---

List concatenation creates a new list with a new identity. When this happens, the old list will eventually be freed from memory by the garbage collector. You'll have to consult the Python documentation to see which methods and operations modify objects in-place and which overwrite objects. A good rule to keep in mind is that if you see a literal in the source code, such as `['rat']` in the previous example, Python will most likely create a new object. A method that is called on the object, such as `append()`, often modifies the object in-place.

Assignment is simpler for objects of immutable data types like integers, strings, or tuples. For example, enter the following into the interactive shell: